



Sony invests in Discord

Sony invested in Discord and the companies plan on integrating the service with PlayStation in 2022. <http://digit.in/may21-01>



Goodbye Flash

Microsoft has announced that it is finally killing off Flash in Windows 10 this summer. <http://digit.in/may21-02>

DE-STRUCTURING ARRAYS AND OBJECTS

Obtaining elements from arrays, or properties from objects has been, perhaps, the most common requirement to develop more complex use-cases. And ES6 brought about a manner that is way easier than the age old mechanism of obtaining individual elements from arrays.

```

1 const names = ['Alice', 'Bob', 'Mark', 'Jude'];
2
3 const [firstName, , thirdName] = names;
4 console.log(
5   `The first name in the array is ${firstName}
6   and the third name is ${thirdName}`
7 );

```

```

The first name in the array is Alice and the third
name is Mark
> |

```

We can simply use the square brackets and in it provide the variable names, separated by commas in which we want the individual names to be stored. Another interesting point as observed from the image here is that we had no requirement for the second name which is exactly why we didn't mention any variable and simply left the space blank, essentially highlighting the fact that even though the second name exists in the names array, we don't want it. This is important because in arrays, the ordering of elements matter, quite unlike objects which we shall see soon. We can also return multiple values from a function with the help of an array and the place where de-structuring comes into play would be while invoking the function. A simple prototype of that would be as follows:

```

function helloWorld(location)
{
  return ['Alabama', 'New York', 'Mumbai',
    location];
}

const [place1, place2]=helloWorld('Mexico');

```

We use de-structuring here while invoking the helloworld function

and just as explained, the place1 variable is set to Alabama and place2 is set to New York. Simple enough, we believe?

Similar to arrays, ES6 also provided us with options for object de-structuring and it's not very different from how we de-structure arrays except for a subtle change in syntax where we use braces (think of

it to be absolutely normal because this indeed is the way how we go about creating objects in JavaScript, don't we?) and not square brackets and the other point of difference being we need to have the same variable names as the properties of the object which we shall de-structure. If that sounds

confusing, the example on the right shall throw your doubts into thin air.

We shall try to de-structure the week object and the way we do that is by using the variables mon and tues inside the curly braces. A major point to note is that the name of the variable has to be the same as the property inside the week object, i.e.,

```

> {open: 10, close: 24} 12 21
Uncaught ReferenceError: wed is not defined
at script.js:21
>

```

The mon variable inside the de-structuring assignment and the same mon property inside the week object. It will unpack the week object and find the value for the mon property, which in this case is another object, the value of which is { open:10, close:24}.

Since this is an object as well, we can take this one step further and even de-structure this as has been shown in the case for the property tues. Adhering to nomenclature, we have a property by the name of tues and a variable

by the name of tues and then we use further braces to extract the values of

```

week = {
  mon: {
    open: 10,
    close: 24,
  },
  tues: {
    open: 12,
    close: 21,
  },
  thurs: {
    open: 07,
    close: 15,
  },
};
//de-structruing begins here
const {
  mon,
  tues: { open, close },
} = week;
console.log(mon, open, close);
console.log(wed)

```

open and close from the object of tues, which in this case, is 12 and 21. Note that we have used the same name for the properties as that of the variables in case of open and close as well. As one last take away from this example, note that we are trying to log a variable called wed. This does not exist neither in the week object, nor is it defined in the de-structuring block. We simply get an error in such a case.

SPREADING AND RESTING

With the advent of ES6, we had two operators

that made our lives way easier than before, especially when it came to dealing with iterables. To be very brief, packing and unpacking of values from an iterable is the fundamental working principle behind these operators. Let's take a look at these in some depth.

Spread operator – This modern feature basically take all the values from an array (or any iterable for that manner) and then places the elements one after another separated by commas as if they were being done manually.

What we basically want to do here is to pass three items to the function, since it expects three parameters. The former way of doing that would